OPAL: Does a Learned Executable Library Reduce the Cost of Later Program Induction?

Author: Eric Lam (independent researcher, Toronto)

Abstract

OPAL is a small program-induction learner. It infers hidden rules as typed lambda-calculus programs, keeps solved programs, and periodically abstracts repeated sub-programs into a growing library. We test whether that library helps by comparing it, in paired runs, with an otherwise identical agent that is never allowed to abstract. On a grid world and a dial-panel domain, the library arm used less search time in every paired comparison and learned more rules in most of them, with margins of about 0.4 to 2 rules per seed depending on the configuration. Two results are negative. Retrieving pieces of past solutions, without any abstraction, gave the control arm most of the benefit on the grid. Library entries did not transfer between the two domains. A coding variant, in which local language models propose programs for integer and list functions, showed no acquisition advantage over search alone and no transfer to new compositions. We report these results as bounded: they come from one author's worlds, four seeds per condition, and code that changed between measurement rounds.

1. Introduction

Program induction systems such as DreamCoder learn a library of reusable functions from solved tasks, and the library lowers the cost of solving later tasks in the same domain (Ellis et al., 2021). Follow-up systems changed how abstractions are found and how the library is used, including Stitch's faster anti-unification and LILO's language-model documentation (Bowers et al., 2023; Grand et al., 2023/24). Other work reports that library learning can look helpful until search cost is counted. In a LEGO-Prover case study, lemma reuse was not found and accuracy gains vanished under compute-matched evaluation (arXiv:2504.03048).

OPAL was built to test a narrower claim than general intelligence. Its central prediction is that interactions needed to reach competence should fall as a learned library grows, and that solve time per task should stay flat or fall (falsification criteria F1 and F2 in the architecture document). The experiments here test a weaker, checkable form: holding everything else fixed, does allowing abstraction reduce search cost and increase rules learned?

Our contributions are: (i) paired comparisons of a library agent and a no-abstraction control on two hidden-rule domains, across several rounds of search changes; (ii) an ablation that gives the control agent retrieval of past pieces; (iii) a cross-domain transfer test; and (iv) a generated-rule benchmark for LLM proposers with memory controls and an oracle-fed diagnostic.

2. Method

Hypothesis language and learner. Hypotheses are programs in a typed lambda calculus with a fixed primitive set and S-expression syntax. The learner searches by typed enumeration with observational-equivalence pruning, backward constraints, and a split driver that recursively solves sub-problems. A stopping rule declares a task learned when the best program predicts at least 95% of next states on a held-out set; a second, never-used trajectory gives the reported test accuracy.

Worlds. The grid world has a single agent, four movement actions, and a curriculum of 12 hidden rules (free movement, solid blocking, gravity, push, walls-only blocking, doors). The dial domain has a panel of dials with verbs and attributes, with 10 rules in the first curriculum and 14 in later diagnostics. Layouts vary by seed, and interactions are spread over three layouts in most runs.

Library agent and control. After each task the library agent refits grammar weights from usage and runs anti-unification abstraction in a sleep phase, adopting an entry only if it reduces total description length by more than a threshold. The control runs the same code with sleep removed. It keeps its solutions and grammar weights, so the only difference is abstraction.

Metrics. We report rules learned (stopping criterion reached), rules at or above 95% accuracy on the independent trajectory, and lifetime search seconds. Each condition uses four seeds. Development used grid seeds 0 to 3 throughout. The one fresh-seed grid test used rule orders from seeds 4 to 7 with a frozen solver.

Coding variant. A local language model proposes programs in a restricted integer, Boolean, and list language. Each proposal is checked against public examples; independent tests are held out and never enter prompts, repair, or selection. The generated benchmark uses three backends (Gemma 3 4B, Qwen 2.5 7B, and no LLM) and three memory controls (fresh, retained experience, and retained experience with abstraction). It has six acquisition tasks and three composition tasks per seed, across seeds 17, 29, and 43. Per-call token and search caps are matched; total compute is not.

3. Experiments and Results

3.1 Library versus paired control

Setting	Library	Control	Notes
Grid, enumeration-only search, seeds 0–3: rules learned (search s)	4.8 (451 s)	2.5 (573 s)	Effect bimodal across seeds
Grid, stronger search (D32): rules ≥95% on test (search s)	8.5 (200 s)	6.5 (489 s)	
Grid, second round: rules ≥95% on test (search s)	9.0 (177 s)	8.0 (478 s)	
Grid, three layouts: rules ≥95% on test (search s)	9.5 (197 s)	9.0 (459 s)	Single-layout: 9.0 vs 7.8
Grid, unseen orders, frozen solver, seeds 4–7: rules ≥95% on test (search s)	9.50 (242 s)	8.25 (567 s)	Fresh seeds
Dial, first curriculum: rules learned (search s)	2.0 (931 s)	0.8 (1196 s)	Machines domain
Dial, sixth round: rules learned / ≥95% on test (search s)	2.8 / 3.0 (1019 s)	2.0 / 2.0 (1295 s)	On-only rule: 3 of 4 vs 0 of 4
Dial, split driver, 20 s per task: rules learned / ≥95% (search s)	4.2 / 4.5 (612 s)	3.8 / 4.2 (792 s)	

The library arm had lower search seconds in every row. It learned more rules or reached more rules at 95% in most rows. The effect was larger on the dial domain's on-only turn rule, which the control learned on no seed, and on the unseen grid orders. Rules that depend on a fold over occupants (walls-only blocking, panel total) and boundary rules (clamp, wrap) remained unsolved by both arms.

Two stepping-stone experiments did not solve the target colour-comparison rule. In the first pass, with three intermediate rules, it was learned in 1 of 12 runs for the library arm and 0 of 12 for the control. In the second pass, with four intermediate rules, it was learned in 1 of 12 runs for both arms.

3.2 Retrieval ablation

Retrieving every closed sub-term of the three most similar past solutions, with no model and no training, was added to both arms.

Setting	Library	Control
Dial, without retrieval: rules learned / ≥95% (search s)	2.8 / 3.0 (1019 s)	2.0 / 2.0 (1295 s)
Dial, with retrieval: rules learned / ≥95% (search s)	3.0 / 3.0 (964 s)	2.0 / 2.0 (1277 s)
Grid, without retrieval: rules learned / ≥95% (search s)	9.0 / 9.5 (206 s)	8.2 / 8.8 (480 s)
Grid, with retrieval: rules learned / ≥95% (search s)	9.0 / 9.5 (201 s)	9.8 / 10.2 (382 s)

On the dial domain, retrieval raised the control's clamp accuracy from 0.50 to 0.90 and decay accuracy from 0.28 to 0.75, without any abstraction. On the grid, retrieval-only control learned more rules than the library arm. The library's advantage on grid search seconds remained (201 s against 382 s).

3.3 Other controls

Test	Result	Verdict
Reuse with challenge, grid: rules learned / ≥95% (search s)	9.0 / 9.5 (199 s) vs reuse off 9.0 / 9.5 (197 s)	Null
90 vs 45 interactions, dial library: clamp / wrap test	0.79 / 0.65 in both	Null
Min and max primitives, dial, seeds 0 and 1: on-only turn	0.88 on both seeds (baseline 1.00)	Negative
Cross-domain transfer, dial after grid: rules learned / ≥95% (search s)	4.00 / 4.25 (611 s) vs alone 4.00 / 4.25 (613 s)	Zero effect
Cross-domain transfer, grid after dial: rules learned / ≥95% (search s)	9.00 / 9.50 (146 s) vs alone 9.00 / 9.50 (153 s)	Zero effect
Goal-directed agent loop vs harness, dial: rules learned / ≥95% (search s)	4.00 / 4.25 (1239 s) vs harness 4.25 / 4.50 (612 s)	Negative against the harness

3.4 Coding variant

In a single integration run on six functions, every arm passed all independent tests: LLM-only 6 of 6 (7 calls, 236 generated tokens), LLM with fallback search 6 of 6 (7 calls, 236 tokens), and LLM with search and library 6 of 6 (7 calls, 208 tokens, one abstraction). This run shows the pipeline works. It is not evidence of an advantage.

In the generated benchmark, tasks passing all 64 final examples (out of 18 acquisition tasks per cell):

Backend	Fresh	Retained experience	Experience + abstraction
Gemma 3 4B	2	4	4
Qwen 2.5 7B	2	4	4
No LLM (search only)	2	4	4

No library entries were adopted in any main-experiment run. Transfer to three new compositions passed 0 of 9 in every backend and control. An oracle-fed diagnostic, which supplied six correct training programs, found a consolidated abstraction and solved 3 of 3 compositions, against 0 of 3 for cold search and 0 of 3 for the same programs without abstraction. A probe of H fluency found that Qwen 2.5 7B wrote correct programs for two simple requests, while Gemma 3 4B did not.

4. Discussion

The library helps the search in these worlds. Across rounds, the library arm had lower search time than its paired control in every comparison, and it learned more rules in most. The later results show that the mechanism of the gain is not simply abstraction. When the control was given retrieval of past pieces, it matched or exceeded the library on learned rules on the grid. The library arm's remaining advantage was search time. This suggests that reuse of partial solutions is the active ingredient in these domains, and that compressing them into entries is one way to obtain it. Retrieval is another, and it costs less to build.

The transfer results are the clearest negative. Library entries from the dial domain and the grid each named their own verbs and attributes. Nothing was reusable across domains, so the result matches the predicted outcome for a library that is not designed for shared concepts. We did not build a second domain with shared concepts, so this test does not establish whether such a library would transfer.

The coding variant does not support an LLM-as-proposer advantage in our setting. Two local models matched search-only acquisition exactly, and the gap between fresh and memory-using arms was the same with and without a model. The one clear positive result is the oracle-fed diagnostic, and it supplies the correct training programs, so it shows what a library can do once the ingredients exist, not that OPAL can find them.

The unsolved rules share a pattern. Clamp, wrap, and decay need boundary tests on the next value, and the linked rule needs a neighbour accessor that is beyond the search's open-term cap. More evidence did not change the description-length choice for clamp and wrap. Adding min and max made on-only worse. Each of these points to representation cost or proposal quality, not to evidence alone.

These findings are consistent with the known risk in library learning: a library can look helpful until search cost is measured (LEGO-Prover case study). OPAL's search-time results are measured directly, and they are the most consistent positive result. The learned-rule counts are small (two to ten rules per curriculum) and the margins are about one rule.

5. Limitations

- Several search and driver changes were made between measurement rounds. The documentation states that search seconds are not comparable across rounds, so comparisons are made within a round.
- Grid seeds 0 to 3 were used for development. The only fresh-seed grid comparison is the frozen-solver run on seeds 4 to 7.
- Four seeds per condition, with descriptive reporting and no significance tests.
- The F2 criterion (solve time flat or falling as the library grows) was not tested as a per-task curve against library size. We report aggregate search seconds only.
- The grid's first enumeration-only result was bimodal, with the library effect present on two of four seeds.
- An intermittent access violation in the enumerator occurred in concurrent batches. One control run died four times and is missing from one table.
- The documents disagree on one number: the machines control is 1.0 rules learned and about 1180 s in DECISIONS.md, and 0.8 and 1196 s in RESULTS.md. We report the RESULTS.md values; the reason for the difference is not recorded.
- The worlds and rule families are the author's design. The coding language is a restricted typed subset, and "LLM-only" there does not mean unrestricted Python generation.
- The autonomous coding experiment has protocol and run files but no written results, so we do not report its outcome.

6. Conclusion

In paired comparisons on two hidden-rule domains, a learned executable library reduced search time relative to an identical control that cannot abstract. The library also raised the number of rules learned in most comparisons, by about 0.4 to 2 rules per seed. Retrieval of past partial solutions gave a control arm most of the same benefit on the grid, so the mechanism is at least partly reuse of pieces rather than abstraction alone. Library entries did not transfer across domains. The hard boundary rules remained unsolved, and LLM proposers in a restricted coding language matched search alone. The results support a narrow claim about search-cost reduction within one domain, and they do not establish general program induction or cross-domain transfer.

References

- Ellis et al. (2021). DreamCoder. PLDI 2021. https://arxiv.org/abs/2006.08381
- Bowers et al. (2023). Stitch. POPL 2023. https://arxiv.org/abs/2211.16605
- Grand et al. (2023/24). LILO. https://arxiv.org/abs/2310.19791
- Wong et al. (2021). LAPS. ICML 2021. https://arxiv.org/abs/2106.11053
- Schmidhuber (2004). Optimal Ordered Problem Solver (OOPS). Machine Learning. https://sferics.idsia.ch/pub/juergen/oopsmlj.pdf
- Schmidhuber (2010). Formal Theory of Fun and Creativity. IEEE TAMD. https://people.idsia.ch/~juergen/ieeecreative.pdf
- Tsividis, Loula, Tenenbaum, Gershman et al. (EMPA, theory-based RL agent). https://arxiv.org/abs/2107.12544
- LLM Library Learning Fails: A LEGO-Prover Case Study. https://arxiv.org/abs/2504.03048
- On the Measure of Intelligence. https://arxiv.org/abs/1911.01547
- Levels of AGI. https://arxiv.org/abs/2311.02462